

7. 功能测试

功能是系统级质量保障的基础！

目 录

CONTENTS

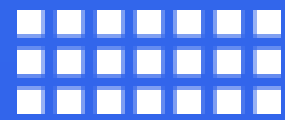
1

功能测试初步

2

功能测试进阶

7.1 功能测试初步



01

功能测试简介



功能测试简介

什么是功能测试？

功能测试是一种验证软件或应用程序是否按预期工作的方法，关注结果是否正确而非内部处理过程。通过为每个功能提供适当的输入，并验证输出是否符合预定义要求，确保系统功能特性满足最终用户的需求。为了评估测试的完整性，功能测试使用功能需求点覆盖率，通过分析已执行的测试来确保所有需求被覆盖。

功能测试的目标

- 验证功能正确性：确保每个功能按预期工作，满足用户需求。
- 覆盖功能需求：通过测试用例确保所有功能需求被验证。
- 保障系统稳定性：通过持续测试确保系统在复杂流程中无错误。



功能测试简介

功能测试的基本流程

1. 确定功能需求：明确需要测试的软件功能需求。
2. 设计测试用例：根据功能需求设计测试场景和步骤。
3. 执行测试与验证：执行测试用例，观察系统行为是否符合预期。

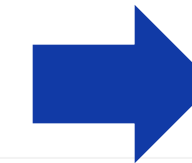
示例：购物车功能测试

- 需求1：将商品添加到购物车。

测试步骤：尝试添加商品，验证是否成功添加且数量更新。

- 需求2：结账流程无错误。

测试步骤：单击结账按钮，输入有效付款和运输信息，验证订单是否成功下达并重定向到确认页面。



测试用例设计的复杂性与扩展

功能测试的基本流程

1. 多样性测试设计：考虑不同的输入、用户场景和系统环境，确保功能在各种情况下均能正常工作。
2. 故障假设测试设计：基于可能出现的故障和异常情况进行测试，确保系统能够正确处理问题。
3. 多功能结合测试：当一次测试多个相关功能时，功能测试的复杂性会增加，需综合考虑交互场景。

示例：超市管理系统“添加账户”功能测试用例设计

- TC1 - 正常账户
- TC2 - 非法用户名
- TC3 - 重复用户名
- TC4 - 非法密码
- TC5 - 密码不一致

思维导图

思维导图的作用与优势

- 帮助测试人员更好地组织测试用例和测试过程。
- 将软件功能点分解为子功能点，明确它们之间的关系和依赖性。
- 提高测试用例的全面性和系统性。
- 便于测试人员理解测试需求，优化测试管理。

实操练习：

- 图7.1展示超市管理系统“基础信息管理”模块四个子模块，其中“系统用户管理”子模块细分为三个功能点，文中对“添加用户”功能点进行了测试用例设计，其他模块相关内容留给读者练习。

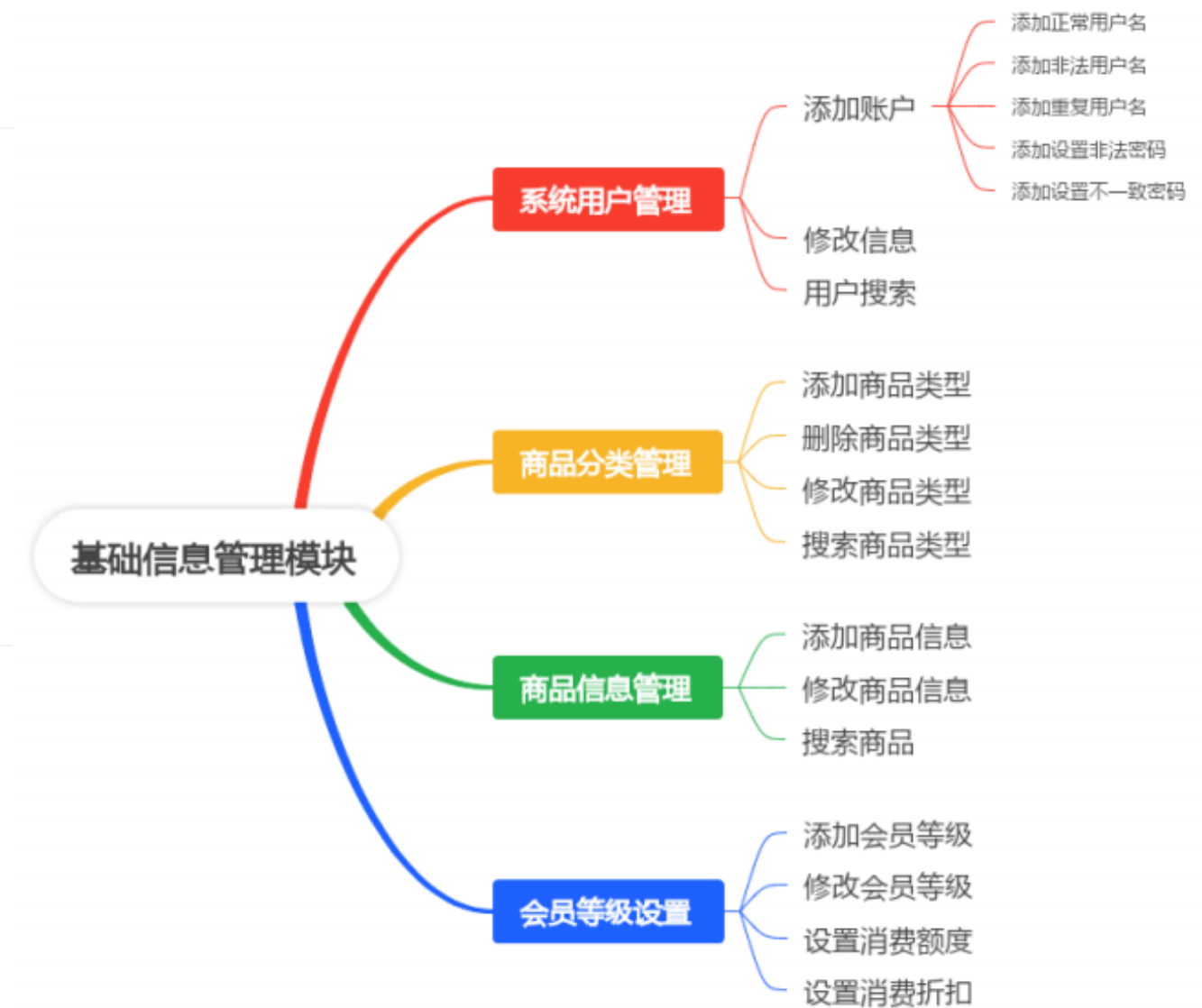


图 7.1: 采用思维导图进行功能测试

功能测试的最佳实践

功能测试的测试过程

功能测试在软件开发阶段是评估过程中非常宝贵的部分，它能够验证产品在发布前是否处于正常工作状态。由于功能测试是从最终用户的角度执行的，开发团队可以创建反映实际用例的测试场景，从而确保产品满足用户需求。

功能测试的最佳实践

- 优先级测试
- 可重用测试
- 拥抱自动化
- 尽可能记录

功能测试与非功能测试

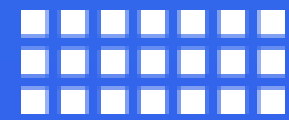
功能测试与非功能测试概述

功能测试和非功能测试是软件测试的两大核心支柱，它们在测试目标、关注点和应用阶段上存在显著差异，但二者相辅相成，共同确保软件的质量和可靠性。

功能测试与非功能测试的区别

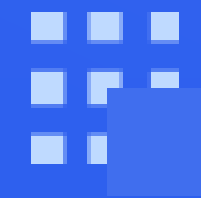
表 7.1: 功能测试和非功能测试

内容	功能测试	非功能测试
关注	软件是否能执行预期功能	评估软件如何执行预期功能
目标	确认正确性和功能性	验证性能，安全性、可用性等
方法	通常黑盒测试	包括黑盒测试和白盒测试
阶段	通常在整个开发过程中进行	通常在功能测试完成后进行
要点	对确保软件过程更为关键	关注于优化和提升用户体验
维护	随着应用需求变化可能频繁更新	通常需要专门的工具和专业知识



02

功能测试工具



自动化测试概述

自动化测试的核心应用场景

- 软件交付前：全面功能测试、兼容性测试。
- 软件交付后：回归测试、持续验证。

自动化功能测试的优势

- 可重复性
- 高效性
- 全面性



POSTMAN



功能测试工具的作用与用途

功能测试工具介绍

功能测试工具是自动化测试的重要支撑，专注于评估软件应用程序的特定功能，包括用户界面、API、数据库、安全性和整体操作行为。核心功能包括：模拟用户交互，验证软件功能是否按预期执行、根据定义的需求评估应用程序的功能表现。

好处与用途

- 提高效率：自动化测试任务，减少手动测试工作量。
- 提高准确性：减少人为错误，确保测试结果可靠。
- 全面覆盖：确保所有功能点都经过测试，提升软件质量。
- 支持持续集成和持续交付（CI/CD），快速反馈测试结果。
- 维护软件开发和质量保证流程的高标准，确保软件可靠性。

Selenium: Web 浏览器自动化测试框架

什么是Selenium?

Selenium 是一个功能强大的开源测试自动化框架，专注于 Web 浏览器的自动化测试。它封装了一系列工具和库，支持跨浏览器测试、动态组件测试以及与多种编程语言的集成，是自动化功能测试的常用工具。

Selenium 的主要组件

- Selenium WebDriver: 提供特定于语言的绑定，直接驱动浏览器，实现原生方式的浏览器自动化。是 W3C 推荐标准，支持创建健壮的回归自动化测试套件。
- Selenium IDE: 是一个浏览器插件，支持录制和回放测试脚本，适合快速创建简单的测试用例。
- Selenium Grid : 支持在多个环境和设备上并行运行测试脚本，提高测试效率，适合大规模测试需求。

Selenium WebDriver 的核心特点与应用场景

Selenium 的核心特点

- 跨浏览器测试
- 多语言支持
- 动态组件测试
- 可扩展性
- 客户端-服务器架构

Selenium WebDriver 的优势

- 回归测试
- 跨浏览器测试
- 分布式测试

示例：使用 Selenium 进行登录功能测试的 Python 脚本

前提条件

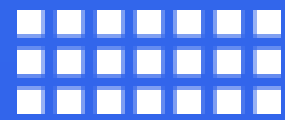
- 安装 Selenium 库
- 替换实际URL
- WebDriver 配置

脚本示例

表 7.2: Selenium 测试脚本-用户名密码登录

```
1 # 创建 WebDriver 实例
2 driver = webdriver.Chrome() # 这里使用Chrome, 也可以根据需要使用其他浏览器
3 # 打开登录页面
4 driver.get("http://example.com/login") # 替换为你的登录页面URL
5 # 等待页面加载完成
6 driver.implicitly_wait(10) # 隐式等待, 最多等待10秒
7 # 找到用户名和密码输入框
8 username_input = driver.find_element(By.ID, "username")
9 password_input = driver.find_element(By.ID, "password")
10 # 清除输入框内容 (如果有的话)
11 username_input.clear()
12 password_input.clear()
13 # 输入用户名和密码
14 username_input.send_keys("your_username") # 替换为实际用户名
15 password_input.send_keys("your_password") # 替换为实际密码
16 # 找到登录按钮并点击
17 login_button = driver.find_element(By.ID, "login-btn") # 使用实际的定位方式替换
18 login_button.click()
19 # 等待登录成功后的页面跳转或提示
20 try:
21     # 这里假设登录成功后会有一个欢迎消息, 可以根据实际情况调整
22     welcome_message = WebDriverWait(driver, 10).until(
23         EC.presence_of_element_located((By.ID, "welcome-message"))
24     )
25     print("登录成功, 欢迎消息:", welcome_message.text)
26 finally:
27     # 关闭浏览器窗口
28     driver.quit()
```

7.2 功能测试进阶



01

多样性功能测试



等价类划分在软件测试中的应用及基本原则

等价类划分的概念

等价类划分是软件测试中基于功能点将输入域划分为有效与无效两个子集的方法，同一子集的输入数据对程序功能实现等效。

- 有效等价类：符合规格说明的合理输入，用于验证功能正确性。
- 无效等价类：不符合规格说明的异常输入，用于测试程序的容错能力。

等价类划分的两个基本原则

- 完备性：等价类的并集应涵盖整个输入域或功能点全集，确保所有可能的输入数据都被考虑在内，没有遗漏。
- 无冗余：等价类之间互不相交，避免重复测试相同的数据集，提高测试效率。

“添加用户”功能的等价类划分策略

有效等价类

- (1) 合法的用户名;
- (2) 合法的密码;
- (3) 合法的联系信息;
- (4) 合法的角色权限。

无效等价类

- (1) 非法的用户名;
- (2) 非法的密码;
- (3) 非法的联系信息;
- (4) 非法的角色权限。

三角形程序 Triangle 的等价类划分与测试设计改进

示例需求

在三角形程序 Triangle 中，要求输入三角形的三个边长：a, b, c。程序功能要求：（1）若是等边三角形，则输出“等边三角形”；（2）若是等腰三角形，则输出“等腰三角形”；（3）若是普通三角形，则输出“普通三角形”；（4）若不是三角形，则输出“无效三角形”。

表 7.3: Triangle 程序功能测试需求划分

划分类别	需求编号	有效划分	需求编号	无效划分
普通	r1	$a + b > c$	r8	$a + b \leq c$
普通	r2	$a + c > b$	r9	$a + c \leq b$
普通	r3	$b + c > a$	r10	$b + c \leq a$
等腰	r4	$a = b$	r11	$a \neq b$
等腰	r5	$b = c$	r12	$b \neq c$
等腰	r6	$a = c$	r13	$a \neq c$
等边	r7	$a = b = c$	r14-r16	$a \neq b$ 或 $b \neq c$ 或 $a \neq c$
其他			r17-r19	a 或 b 或 c 为 null
其他			r20-r22	a 或 b 或 c 为非正数
其他			r23-r25	a 或 b 或 c 为非数字

表 7.4: Triangle 程序功能测试用例设计

测试编号	测试输入	需求编号	预期输出
t1	[2,3,4]	r1-r3, r11-r16	普通三角形
t2	[1,1,3]	r4, r8	无效三角形
t3	[2,0,0]	r5, r10, r20-r22	无效三角形
t4	[3,2,3]	r6, r9	等腰三角形
t5	[2,2,2]	r7	等边三角形
t6	[null,2,2]	r17	无效三角形
t7	[:,2,2]	r23	无效三角形

因果图在测试用例设计中的应用

什么是因果图？

因果图是一种系统化的测试设计技术，通过分析程序输入（原因）与输出（效果）之间的逻辑关系，帮助选择高效的测试集。

因果图的构建

1. 确定原因和效果
2. 构建因果图
3. 处理约束
4. 生成决策表
5. 设计测试用例

因果图的优势

- 系统化分析
- 避免组合爆炸
- 可视化逻辑关系
- 高效生成测试集

因果逻辑关系的四种类型

因果逻辑关系的四种类型概述

在因果图分析中，输入（原因）与输出（效果）之间的逻辑关系可以分为四种基本类型：恒等关系（EQ）、逻辑非（NOT）、逻辑或（OR）和逻辑与（AND）。

恒等关系（EQ）

定义：如果原因 A 为 1（真），则效果 B 为 1；否则 B 为 0（假）。

逻辑非（NOT）

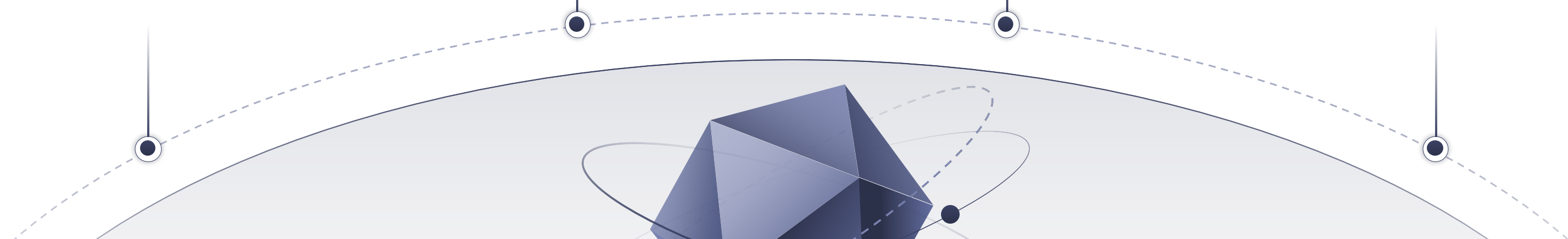
定义：如果原因 A 为 1，则效果 B 为 0；否则 B 为 1。

逻辑或（OR）

定义：如果原因 A 或原因 B 为 1，则效果 C 为 1；否则 C 为 0。

逻辑与（AND）

定义：如果原因 A 和原因 B 都为 1，则效果 C 为 1；否则 C 为 0。



三角形程序的因果图分析与测试设计

因果图分析三角形程序

通过因果图分析，系统化地识别了三角形程序的输入条件（原因）与输出结果（效果）之间的逻辑关系，并生成了决策表和测试用例。

表 7.5: 三角形程序 Triangle 决策表

条件	r_1	r_2	r_3	r_4	r_5	r_6	r_7	r_8	r_9	r_{10}	r_{11}
$c_1 : a < b + c?$	0	1	1	1	1	1	1	1	1	1	1
$c_2 : b < a + c?$	×	0	1	1	1	1	1	1	1	1	1
$c_3 : c < a + b?$	×	×	0	1	1	1	1	1	1	1	1
$c_4 : a = b?$	×	×	×	1	1	1	1	0	0	0	0
$c_5 : a = c?$	×	×	×	1	1	0	0	1	1	0	0
$c_6 : b = c?$	×	×	×	1	0	1	0	1	0	1	0
e_1 : 无效三角形	1	1	1								
e_2 : 普通三角形							1				1
e_3 : 等腰三角形					×	×		×	1	1	
e_4 : 等边三角形				1							

表 7.6: 三角形程序 Triangle 测试用例

编号 r_i	a	b	c	预期输出
1	4	1	2	无效三角形
2	1	4	2	无效三角形
3	1	2	4	无效三角形
4	5	5	5	等边三角形
5				无法满足
6				无法满足
7	2	2	3	等腰三角形
8				无法满足
9	2	3	2	等腰三角形
10	3	2	2	等腰三角形
11	3	4	5	普通三角形

类别划分 (Category-Partition, CP)

类别划分 (Category-Partition, CP) 概述

类别划分测试方法是一种系统化的功能测试技术，通过识别参数和环境变量，将其划分为类别，并为每个类别生成选项，最终组合成测试框架。

类别划分法的核心步骤

1. 识别参数和环境变量
2. 定义类别
3. 为类别生成选项
4. 组合选项生成测试框架
5. 生成测试用例

类别划分法的优势

- 系统化
- 高效性
- 可扩展性
- 规范驱动

使用类别划分 (Category-Partition, CP) 方法对三角形程序进行功能测试

示例需求

以一个简单的三角形分类程序为例，该程序根据三边长度判断三角形是等边、等腰还是一般三角形，或者不是三角形。输入属性有三个：边 a、边 b 和边 c 的长度。

练习：

- 使用类别划分 (Category-Partition, CP) 方法对三角形程序进行功能测试。系统化地识别三角形程序的输入属性，定义类别和选项，并基于这些划分，生成覆盖有效和无效输入的测试用例。

使用类别划分（Category-Partition, CP）方法对三角形程序进行功能测试

示例需求

假设我们要测试一个商业银行贷款程序，该程序根据客户的就业状态、就业类型、工作类型、月薪、申请人类型和信用卡类型来决定是否批准贷款申请。

类别和选项划分

表 7.7: 贷款功能的类别和选项划分

就业状态	” 就业” 或” 失业”
就业类型	” 自雇” 或” 受雇于他人”
工作类型	” 永久” 或” 临时”
月薪 S	$0 < S \leq 2000$, $2000 < S \leq 3,000$, 或 $S > 3,000$
申请人类型	” 持卡人” 或” 非持卡人”
信用卡类型	” 金卡” 或” 普通卡”
普通卡信用额度	”2000” 或”3000”. 金卡无信用额度限制.

根据选项之间的关系，构造以下测试框

- 框1: 就业 + 自雇 + 永久 + 月薪 > 2000 + 持卡人 + 金卡
- 框2: 就业 + 受雇于他人 + 永久 + 月薪 > 2000 + 非持卡人
- 框3: 就业 + 自雇 + 临时 + 月薪 ≤ 2000 + 持卡人 + 金卡
- 框4: 就业 + 受雇于他人 + 临时 + 月薪 ≤ 2000 + 非持卡人
- 框5: 失业 + 自雇 + 永久 + 月薪 > 2000 + 持卡人 + 金卡
- 框6: 失业 + 受雇于他人 + 永久 + 月薪 > 2000 + 非持卡人
- 框7: 失业 + 自雇 + 临时 + 月薪 ≤ 2000 + 持卡人 + 金卡
- 框8: 失业 + 受雇于他人 + 临时 + 月薪 ≤ 2000 + 非持卡人

测试框及其完整性定义

定义 7.1: 测试框及其完整性

一个测试框 B 是一组选项。如果每当从 B 中的每个选项中选择一个元素时，将形成一个独立的输入，则称测试框 B 是完整的。否则，成为不完整的。

➤ 测试框 B :

测试框 B 是一组选项的集合，每个选项代表一个类别中的一个分区或等价类。

➤ 完整性:

如果从测试框 B 中的每个选项中各选择一个元素时，能够形成一个独立的输入（即一个有效的测试用例），则称测试框 B 是完整的。

如果无法形成独立的输入（例如，某些选项之间存在冲突或约束），则称测试框 B 是不完整的。

与选项相关的完整测试框架集定义

定义 7.2: 与选项相关的完整测试框集

让 TF 表示所有完整测试框的集合。给定任何选项 x ，将与 x 相关的完整测试框集定义为 $TF(x) = \{B \in TF : x \in B\}$ 。当且仅当 $TF(x)$ 非空时，选项 x 才有效。♣

➤ 完整测试框集 TF :

TF 是所有完整测试框的集合。每个完整测试框 $B \in TF$ 是一组选项，且从 B 中每个选项选择一个元素时，能形成一个独立的输入（即测试框 B 是完整的）。

➤ 与选项 x 相关的完整测试框集 $TF(x)$:

对于任意选项 x ，定义 $TF(x)$ 为所有包含 x 的完整测试框的集合，即：

$$TF(x) = \{B \in TF : x \in B\}$$

➤ 选项 x 的有效性:

当且仅当 $TF(x)$ 非空时，选项 x 才有效。也就是说，如果至少存在一个完整测试框 B 包含选项 x ，则 x 是有效的；否则， x 是无效的。

两个选项之间的关系

定义 7.3: 两个选项之间的关系

给定任何有效选项 x ，它与另一个有效选项 y 的关系，记为 $x \mapsto y$ ，根据以下三个关系运算符之一定义：

- 当且仅当 $TF(x) \subseteq TF(y)$ 时， x 完全嵌入在 y 中，记为 $x \sqsubset y$ 。
- 当且仅当 $TF(x) \not\subseteq TF(y)$ 和 $TF(x) \cap TF(y) \neq \emptyset$ 时， x 部分嵌入在 y 中，记为 $x \sqsubset_P y$ 。
- 当且仅当 $TF(x) \cap TF(y) = \emptyset$ 时， x 不嵌入 y ，记为 $x \not\sqsubset y$ 。

➤ 完全嵌入 ($x \sqsubset y$)：

y 比 x 更通用，因为所有包含 x 的测试框也包含 y 。

➤ 部分嵌入 ($x \sqsubset_P y$)：

x 和 y 在某些测试框中共同出现，但并非所有情况。

➤ 不嵌入 ($x \not\sqsubset y$)：

x 和 y 互不相关，无法在同一测试框中同时出现。

类别选项推理性质

定理 7.1: 类别选项推理性质

x 、 y 和 z 为有效选项，类别选项具有以下推理性质：

- (1) 非嵌入对称性： $x \sqsubset y$ 当且仅当 $y \sqsupset x$ 。
- (2) 完全和部分嵌入反向性：1) 如果 $x \sqsubset y$ ，则 $y \sqsubset x$ 或 $y \sqsubset_P x$ 。2) 如果 $x \sqsubset_P y$ ，则 $y \sqsubset x$ 或 $y \sqsubset_P x$ 。
- (3) 完全嵌入传递性：1) 如果 $x \sqsubset y$ 和 $y \sqsubset z$ ，那么 $x \sqsubset z$ 。2) 如果 $x \sqsubset y$ 和 $x \sqsubset z$ ，那么 $y \sqsubset z$ 或 $y \sqsubset_P z$ 。
- (4) 完全嵌入和非嵌入传递性：如果 $x \sqsubset y$ 和 $y \# z$ ，那么 $x \# z$ 。2) 如果 $x \sqsubset y$ 和 $x \# z$ ，则 $y \sqsubset_P z$ 或 $y \# z$ 。
- (5) 完全嵌入和非嵌入传递性：1) 如果 $x \sqsubset z$ 和 $y \# z$ ，那么 $x \# y$ 。2) 如果 $y \sqsubset z$ 和 $x \# y$ ，则 $z \sqsubset_P x$ 或 $z \# x$ 。
- (6) 完全嵌入和部分嵌入传递性：1) 如果 $x \sqsubset y$ 和 $x \sqsubset_P z$ 那么 $y \sqsubset_P z$ 。2) 如果 $x \sqsubset z$ 和 $y \sqsubset_P z$ ，则 $y \sqsubset_P x$ 或 $y \# x$ 。3) 如果 $y \sqsubset z$ 和 $x \sqsubset_P y$ ，那么 $z \sqsubset x$ 或 $z \sqsubset_P x$ 。
- (7) 部分嵌入和非嵌入传递性：1) 如果 $x \sqsubset_P y$ 和 $y \# z$ ，则 $x \sqsubset_P z$ 或 $x \# z$ 。
2) 如果 $x \sqsubset_P y$ 和 $x \# z$ ，则 $y \sqsubset_P z$ 或 $y \# z$ 。
- (8) 部分嵌入和非嵌入传递性：1) 如果 $x \sqsubset_P z$ 和 $y \# z$ ，则 $x \sqsubset_P y$ 或 $x \# y$ 。
2) 如果 $y \sqsubset_P z$ 和 $x \# y$ ，则 $z \sqsubset_P x$ 或 $z \# x$ 。



示例：商业银行贷款程序

选项关系

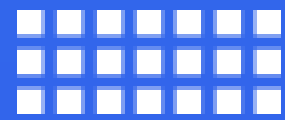
- 就业状态 和 就业类型 之间没有直接关系。
- 持卡人和 非持卡人 选项不能同时出现，它们之间是互斥关系。
- 金卡 和 普通卡 选项可以同时出现，它们之间是包容关系。

测试用例

- 通过从每个测试框中选择单个选项，可以生成测试用例。例如，框1中的测试用例可以是：
- 就业 + 自雇 + 永久 + 月薪 > 2000 + 持卡人 + 金卡

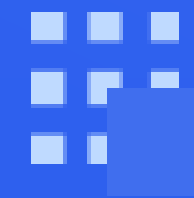
总结

通过将类别和选项应用于功能测试，我们可以有效地管理测试用例，并确保测试用例的全面性和效率。



02

故障假设功能测试



故障假设功能测试概述

什么是基于故障假设功能测试？

基于故障假设的功能测试是一种通过模拟故障场景来评估系统功能性和可靠性的方法。其核心目标是验证系统在故障条件下的表现，确保系统具备足够的容错能力和稳定性。

故障假设的关键步骤和实施要点

1. 测试规划与分析
2. 测试实施方法
3. 测试评估与改进

不同测试水平的变异差异

图7.2展示了在不同测试水平（单元、类、集成和系统）上实施变异分析的差异。每个测试水平关注的焦点、变异方法以及测试目标各有不同。

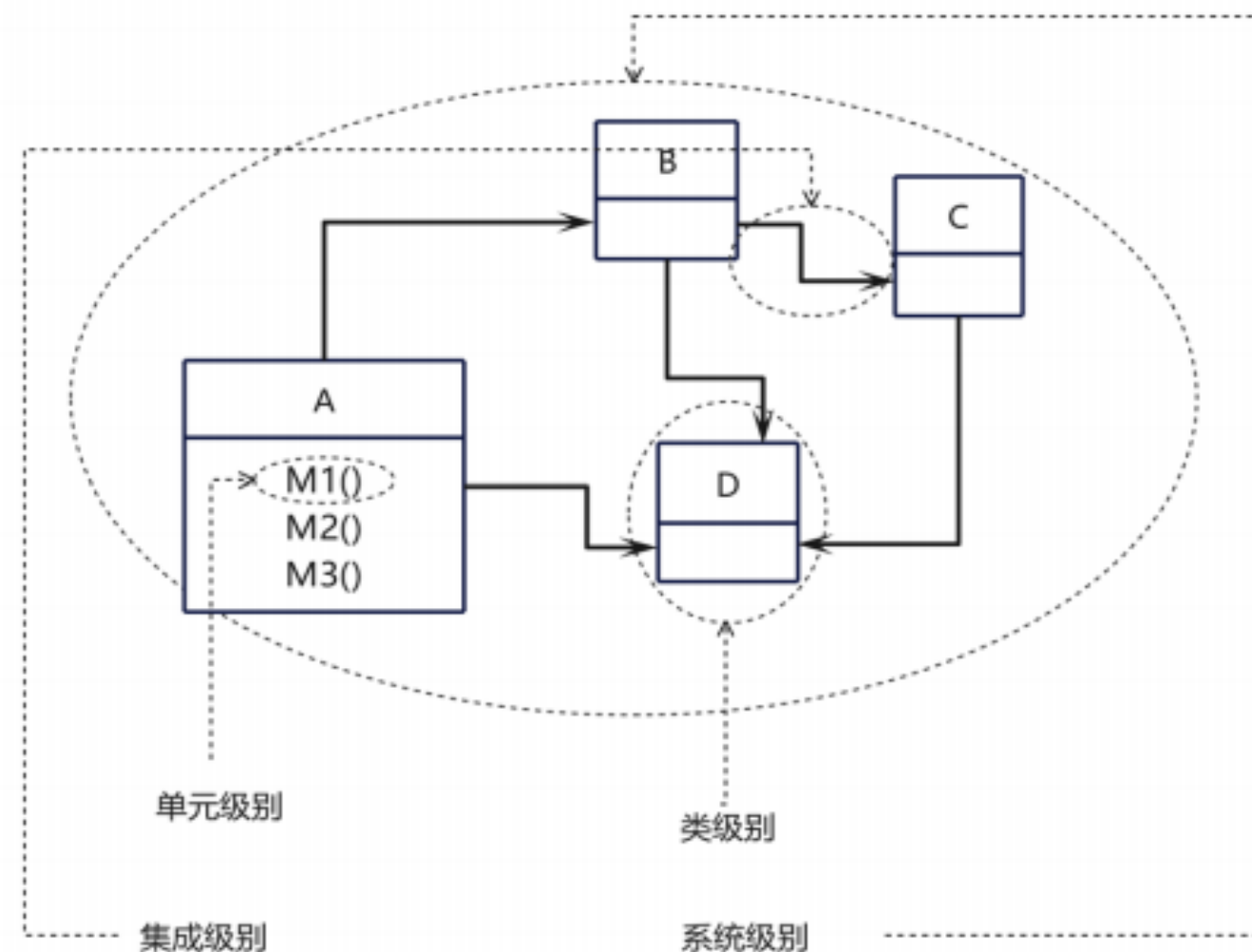


图 7.2: 系统级变异

- 单元级别：关注单个方法的变异和测试，成本低但测试范围有限。
- 类级别：关注整个类及其内部方法的变异和测试，测试粒度中等。
- 集成级别：关注组件之间连接的变异和测试，测试粒度较大。
- 系统级别：关注整个系统的变异和测试，测试粒度最大，能够检测所有元素之间的交互。

系统级别变异的屏蔽概念

什么是变异的屏蔽？

变异的屏蔽概念（mutation shielding）是指变异引入的错误状态或行为在后续执行中被“掩盖”或“纠正”，导致测试用例未能检测到异常，使得变异体未被“杀死”，从而削弱变异测试的有效性。

屏蔽机制

- 带有错误的值可能在后续执行中被新的正确值覆盖，从而消除错误的影响。
- 一个值不正确的变量可能会超出范围，从而丢失其错误值，避免感染其他状态。
- 具有不正确值的对象可能在错误传播之前被销毁，从而阻止错误状态的扩散。

测试的执行需要加载问题的解决办法

系统级变异测试执行痛点

在系统级变异测试中，每次测试的执行都需要加载原始整体系统，这比单元级测试更为复杂。

痛点解决方法

1. 更换整个系统：
 - 实现方式：生成并加载一个完整的变异系统。
2. 更换系统部分组件：
 - 实现方式：仅替换系统中发生变异的组件，而非整个系统。

定义系统级变异的新算子

大多数系统级故障都是相关的单元或组件之间的相互作用以及组成系统的元素的配置。系统测试侧重于顶层功能，而集成测试则侧重于之间的连接软件组件。在系统层面，故障可以与用户界面和用户交互相关。

表 7.8: 调用相关变异算子

CMCR	Compatible Method Calls Replacement	替换一个兼容的方法调用
MOR	Method Overloading Replacement	替换一个重载的方法调用
COI	Calls Order Interchange	交换两个方法调用次序

表 7.9: 界面相关变异算子

GCPI	Graphical Component Position Interchange	交换两个界面组件
GOCOC	Graphical Ordered Component Order Interchange	交换两个界面组件元素
GCD	Graphical Component Deletion	删除一个界面组件
DGCM	Disabled Graphical Component Modification	将某个界面组件从 enable 变为 disable

强变异和弱变异在系统级测试中的区别及其局限性

强变异与弱变异的特点

1. 强变异的特点:

- 强变异需要异常状态通过输出到外部环境或在输入驱动系统的中间状态中传播, 才能被检测到

2. 弱变异的特点:

- 弱变异忽略异常状态的传播, 仅比较程序执行的中间状态。
- 在单元级别, 弱变异已被广泛应用, 因为状态较小且局部化。

故障假设的关键步骤和实施要点

在弱变异中, 执行变异语句后, 程序停止并比较变异体与原始系统的状态。如果异常状态存在于另一个非变异对象, 传统弱变异无法检测到, 导致变异体未被“杀死”¹³。

即使存在异常状态, 另一个对象也可能不受影响, 进一步限制了弱变异的有效性¹³。

强变异和弱变异在检测变异体时的局限性示例

这是一个具有三个类（ClassA、ClassB 和 ClassC）的系统，以及一个测试用例，该测试用例生成了一系列调用这些类方法的序列。系统通过执行 ClassA 的方法 aM1(int x, int y) 来触发一系列方法调用，涉及 ClassB 和 ClassC 的方法，并创建和销毁 ClassC 的对象。

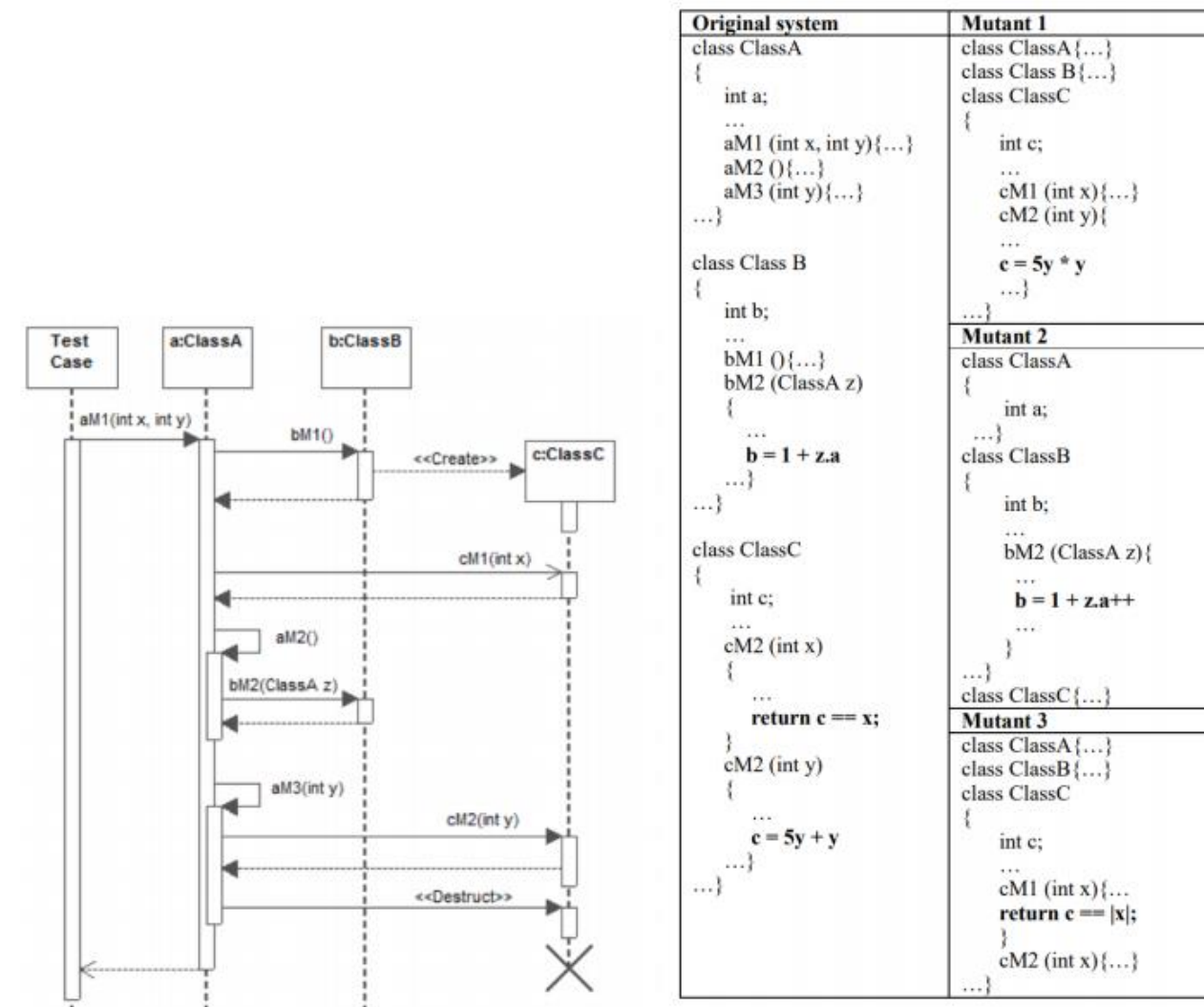


图 7.3: 系统级变异

弹性弱变异方法

弹性弱变异的核心思想

- 弹性弱变异结合了弱变异和强变异的优点，动态决定何时停止执行。
- 它通过在发现异常状态时停止执行，而不是在预设的固定点停止，从而更灵活地检测变异体的影响。

弹性弱变异的执行步骤

步骤 1：执行原始系统时，存储所有执行状态信息。每个方法的状态在方法开始和结束时被记录，系统执行结束后保留完整的状态序列。

步骤 2：执行变异体时，将变异体的状态序列与原始系统的状态序列进行比较。比较点包括方法开始和结束时的状态。

步骤 3：如果在执行变异体期间发现状态差异，立即停止执行，并记录该变异体被杀死。

步骤 4：如果没有发现异常状态，则继续执行直到结束。

弹性弱变异的特点与优势

弹性弱变异的核心思想

- 动态停止：只有在发现异常状态时才停止执行，而不是在预设的固定点停止。
- 多次检查：变异体的状态会在多个时间点和位置进行检查，增加发现异常的机会。
- 跨对象比较：弹性弱变异允许对多个对象的状态进行比较，而不仅仅局限于单个对象。
- 状态跟踪：原始系统的状态序列被存储，并在运行时与变异体的状态序列进行比较，以评估变异体的影响。

弹性弱变异的优势

如果变异体没有立即导致明显的状态异常，允许继续执行到最后输出，从而更全面地评估变异体的影响。通过多次检查和跨对象比较，能够更早地发现异常状态，并在适当的时候终止变异体的执行。